MODUL HANDS-ON

MEMBUAT DASHBOARD PENJUALAN DENGAN STREAMLIT

Anaconda + Anaconda Prompt + Python + Pandas + Plotly

Level: Pemula | **Metode:** Praktik langsung

Modul ini membawa peserta dari nol sampai menghasilkan dashboard penjualan interaktif. VS Code tidak diperlukan; Notepad cukup untuk menulis file Python.

1. TUJUAN PRAKTIKUM

Peserta akan memahami alur data dari CSV → Pandas → analisis → Streamlit → Plotly → dashboard. Fokus modul adalah memahami **mengapa kode digunakan**, bukan hanya menyalin kode.

- Membuat environment Python dengan Anaconda.
- Menjalankan Streamlit melalui Anaconda Prompt.
- Membaca CSV dengan Pandas.
- Membuat KPI dan filter interaktif.
- Membuat grafik line, bar, dan donut.
- Menampilkan tabel dan download data.
- Memahami hubungan antara input user, proses data, dan output dashboard.

Output akhir

Komponen	Hasil
KPI	Penjualan, profit, transaksi, qty, margin
Filter	Tanggal, kategori, channel, wilayah
Visualisasi	Tren bulanan, kategori, top produk, channel
Detail	Tabel transaksi + download CSV

2. PERSIAPAN

Perangkat yang diperlukan:

- Anaconda Navigator/Anaconda Distribution.
- Anaconda Prompt.
- Chrome atau Edge.
- Notepad. VS Code opsional.

File praktikum

File	Fungsi
app.py	Program Streamlit
data_penjualan.csv	600 transaksi dummy
README_dashboard.txt	Petunjuk singkat

3. MEMBUAT ENVIRONMENT

Environment memisahkan library dan Python antar-project. Dengan environment, instalasi untuk dashboard tidak mengganggu project Python lainnya.

Di Anaconda Navigator: Environments → Create → Name: **dashboard** → Python 3.11 → Create.

Kemudian buka Anaconda Prompt:

```
conda env list
conda activate dashboard
python --version
```

Jika berhasil, terminal akan diawali **(dashboard)**.

4. INSTALL LIBRARY

```
pip install streamlit pandas plotly openpyxl
streamlit --version
```

Library	Fungsi
streamlit	Aplikasi dashboard
pandas	Pengolahan data
plotly	Grafik interaktif
openpyxl	Membaca Excel

5. MENYIAPKAN FOLDER

Buat folder project, misalnya D:\Dashboard. Di Anaconda Prompt:

```
mkdir D:\Dashboard
cd D:\Dashboard
```

Letakkan dua file utama dalam folder yang sama:

```
D:\Dashboard
■■■ app.py
■■■ data_penjualan.csv
```

Nama file dan lokasi penting karena kode akan mencari data_penjualan.csv dari folder kerja saat Streamlit dijalankan.

6. STREAMLIT PERTAMA

Buat file dengan Notepad:

```
notepad app.py
```

Masukkan kode berikut:

```
import streamlit as st

st.title("Dashboard Penjualan")
st.write("Ini adalah dashboard pertama saya!")

col1, col2, col3 = st.columns(3)

with col1:
    st.metric("Total Penjualan", "Rp 50.000.000")

with col2:
    st.metric("Total Transaksi", "250")

with col3:
    st.metric("Produk Terjual", "750")
```

Simpan dengan Ctrl+S. Jalankan:

```
python -m streamlit run app.py
```

Buka **http://localhost:8501**. Untuk menghentikan server tekan Ctrl+C di Anaconda Prompt.

7. MEMAHAMI SINTAKS DASAR

Kode	Fungsi
import streamlit as st	Memanggil Streamlit dan memberi alias st.
st.title()	Menampilkan judul besar.
st.write()	Menampilkan teks atau output.
st.columns()	Membagi area halaman menjadi beberapa kolom.
st.metric()	Menampilkan angka KPI.
st.sidebar	Menempatkan komponen pada sidebar.

Setelah memahami contoh sederhana ini, kita mulai membangun dashboard dari data transaksi sungguhan.

8. MEMBACA DATA DENGAN PANDAS

Sebelum membuat filter atau grafik, dashboard harus memiliki sumber data. Pada praktikum ini sumber data adalah **data_penjualan.csv**. Pandas digunakan untuk mengubah file tersebut menjadi DataFrame.

Kode:

```
import pandas as pd

@st.cache_data
def load_data():
    df = pd.read_csv("data_penjualan.csv", parse_dates=["Tanggal"])
    return df

df = load_data()
```

Penjelasan:

- **import pandas as pd** — memanggil Pandas dengan alias pd.
- **pd.read_csv()** — membaca file CSV menjadi DataFrame.
- **parse_dates=['Tanggal']** — memberi tahu Pandas bahwa kolom Tanggal harus diperlakukan sebagai tanggal.
- **df** — nama variabel yang menyimpan tabel data.
- **return df** — mengembalikan DataFrame dari fungsi.
- **df = load_data()** — menjalankan fungsi dan menyimpan hasilnya ke df.

Apa itu DataFrame?

DataFrame dapat dibayangkan sebagai tabel Excel yang dapat diproses menggunakan Python. Dataset praktikum memiliki kolom seperti Tanggal, No_Transaksi, Produk, Kategori, Qty, Harga_Satuan, Penjualan, HPP, Profit, Channel, Wilayah, dan Sales.

Eksplorasi data sementara:

```
st.write(df.head())
st.write(df.shape)
st.write(df.columns)
```

df.head() melihat beberapa baris awal. **df.shape** mengembalikan jumlah baris dan kolom. **df.columns** menampilkan daftar nama kolom. Setelah memahami data, tiga baris eksplorasi ini boleh dihapus.

9. MEMBUAT SIDEBAR FILTER

Filter adalah bagian penting dashboard karena memungkinkan pengguna memilih subset data tanpa mengubah dataset asli. Kita menempatkan filter di sebelah kiri menggunakan **st.sidebar**.

9.1 Membuat judul sidebar

```
st.sidebar.header("■ Filter Dashboard")
```

st.sidebar berarti komponen dibuat di area sidebar. **header()** memberikan judul pada kelompok kontrol filter.

9.2 Menentukan batas tanggal

```
min_date = df["Tanggal"].min().date()
max_date = df["Tanggal"].max().date()
```

Kita tidak ingin menulis tanggal secara manual. **min()** mencari tanggal paling awal dari data, sedangkan **max()** mencari tanggal paling akhir. **.date()** mengambil bagian tanggal tanpa waktu.

Contoh: jika dataset berisi 1 Januari sampai 30 September 2026, maka min_date dan max_date otomatis mengikuti data tersebut.

9.3 Membuat date picker

```
date_range = st.sidebar.date_input(
```

```
    "Periode",
    value=(min_date, max_date),
    min_value=min_date,
    max_value=max_date
)
```

date_input() menghasilkan kontrol pemilihan tanggal. Parameter **value** menentukan nilai awal, sedangkan **min_value** dan **max_value** membatasi pilihan agar tidak keluar dari rentang data.

Mengapa tidak langsung menggunakan tanggal manual?

Karena dashboard harus dapat digunakan kembali ketika dataset berubah. Jika data tahun depan dimasukkan, batas tanggal akan mengikuti data secara otomatis.

9.4 Membuat filter kategori

```
categories = st.sidebar.multiselect(
    "Kategori",
    sorted(df["Kategori"].unique()),
    default=sorted(df["Kategori"].unique())
)
```

Baris pertama membuat komponen multiselect. Argumen kedua adalah daftar pilihan. **df['Kategori'].unique()** mengambil kategori yang tidak duplikat. **sorted()** mengurutkannya. **default=...** membuat seluruh kategori terpilih pada awal dashboard.

Hasilnya, user dapat memilih misalnya Undangan + Souvenir saja. Nilai pilihan tersebut disimpan dalam variabel **categories**.

9.5 Pola yang sama untuk Channel dan Wilayah

```
channels = st.sidebar.multiselect(
    "Channel",
    sorted(df["Channel"].unique()),
    default=sorted(df["Channel"].unique())
)

regions = st.sidebar.multiselect(
    "Wilayah",
    sorted(df["Wilayah"].unique()),
    default=sorted(df["Wilayah"].unique())
)
```

Sekarang dashboard memiliki empat kontrol: periode, kategori, channel, dan wilayah. Yang penting dipahami adalah setiap kontrol **menghasilkan nilai pilihan user** yang akan digunakan pada tahap filtering.

10. FILTERING DATA DENGAN PANDAS

Setelah user memilih filter, kita perlu membuat DataFrame baru yang hanya berisi transaksi yang sesuai. Data asli **df** tetap dipertahankan; hasil filter disimpan sebagai **filtered**.

Kode inti:

```
filtered = df[
    (df["Tanggal"].dt.date >= start_date) &
    (df["Tanggal"].dt.date <= end_date) &
    (df["Kategori"].isin(categories)) &
    (df["Channel"].isin(channels)) &
    (df["Wilayah"].isin(regions))
].copy()
```

Mari pecah menjadi beberapa kondisi:

Kode	Arti
df['Tanggal'].dt.date >= start_date	Tanggal transaksi sama dengan atau setelah tanggal awal.
df['Tanggal'].dt.date <= end_date	Tanggal transaksi sama dengan atau sebelum tanggal akhir.
df['Kategori'].isin(categories)	Kategori termasuk pilihan user.
df['Channel'].isin(channels)	Channel termasuk pilihan user.

Kode	Arti
df['Wilayah'].isin(regions)	Wilayah termasuk pilihan user.

10.1 Mengapa menggunakan tanda &?

Tanda **&** berarti semua kondisi harus terpenuhi atau AND. Misalnya user memilih Souvenir dan Solo, transaksi harus sekaligus berkategori Souvenir *dan* berada di Solo.

10.2 Mengapa menggunakan isin()?

Karena user dapat memilih lebih dari satu nilai. Misalnya categories berisi ['Souvenir', 'Undangan']. **isin(categories)** akan menghasilkan True untuk baris yang kategorinya Souvenir atau Undangan.

10.3 Mengapa ada .copy()?

copy() membuat salinan hasil filter sebagai DataFrame baru. Ini membantu menghindari efek samping ketika nantinya kita melakukan transformasi pada filtered.

Contoh hasil filtering:

Jika awalnya terdapat 600 transaksi, setelah user memilih Januari–Maret + Souvenir + Instagram + Solo, filtered mungkin hanya berisi sebagian transaksi. Semua KPI dan grafik setelah bagian ini harus menggunakan **filtered**, bukan df, agar ikut berubah ketika filter berubah.

11. MEMBUAT KPI OTOMATIS

KPI adalah angka utama yang membantu pengguna memahami kondisi bisnis secara cepat. Karena dashboard bersifat interaktif, KPI harus dihitung dari **filtered**.

11.1 Total Penjualan

```
total_sales = filtered["Penjualan"].sum()
```

Python mengambil kolom Penjualan dari filtered, lalu **sum()** menjumlahkan seluruh nilainya. Jika filter berubah, nilai total_sales otomatis dihitung ulang.

11.2 Total Profit

```
total_profit = filtered["Profit"].sum()
```

Konsepnya sama, tetapi kolom yang dijumlahkan adalah Profit.

11.3 Total produk terjual

```
total_qty = filtered["Qty"].sum()
```

Menjumlahkan quantity seluruh transaksi yang sesuai filter.

11.4 Jumlah transaksi

```
total_transactions = filtered["No_Transaksi"].nunique()
```

Kita menggunakan **nunique()**, bukan sum(), karena yang ingin dihitung adalah jumlah nomor transaksi unik.

11.5 Profit Margin

```
margin = (total_profit / total_sales * 100) if total_sales else 0
```

Rumusnya: **Profit Margin = Profit ÷ Penjualan × 100%**. Bagian **if total_sales else 0** mencegah pembagian dengan nol ketika tidak ada transaksi setelah filter.

11.6 Menampilkan KPI

```
c1, c2, c3, c4, c5 = st.columns(5)

c1.metric("■ Total Penjualan", f"Rp {total_sales:,.0f}")
c2.metric("■ Total Profit", f"Rp {total_profit:,.0f}")
c3.metric("■ Transaksi", f"{total_transactions:,}")
c4.metric("■ Produk Terjual", f"{total_qty:,}")
c5.metric("■ Profit Margin", f"{margin:.1f}%")
```

st.columns(5) membuat lima area sejajar. Variabel c1–c5 mewakili masing-masing kolom. **st.metric()** menampilkan label dan nilai.

Format angka:

:,.0f digunakan untuk angka tanpa desimal dengan pemisah ribuan. **:.1f** digunakan untuk persentase dengan satu angka desimal. Karakter **f** pada string memungkinkan nilai variabel dimasukkan ke dalam teks.

12. MEMBUAT GRAFIK TREN PENJUALAN

Grafik garis cocok untuk melihat perubahan penjualan dari waktu ke waktu. Sebelum membuat grafik, kita perlu mengubah transaksi harian menjadi ringkasan bulanan.

12.1 Membuat kolom Bulan dan melakukan agregasi

```
monthly = (
    filtered.assign(
        Bulan=filtered["Tanggal"].dt.to_period("M").astype(str)
    )
    .groupby("Bulan", as_index=False)["Penjualan"]
    .sum()
)
```

Penjelasan tahap demi tahap:

- **assign()** membuat kolom baru tanpa perlu mengubah struktur utama filtered.
- **dt.to_period('M')** mengubah tanggal menjadi periode bulanan.
- **astype(str)** mengubah periode menjadi teks agar mudah digunakan sebagai label grafik.
- **groupby('Bulan')** mengelompokkan transaksi berdasarkan bulan.
- **['Penjualan'].sum()** menjumlahkan revenue untuk setiap bulan.
- **as_index=False** menjaga Bulan tetap menjadi kolom biasa.

Contoh perubahan data:

Sebelum	Sesudah
01-01-2026 — Rp1 juta	2026-01 — Rp6 juta
05-01-2026 — Rp2 juta	2026-02 — Rp4 juta
10-01-2026 — Rp3 juta	...

12.2 Membuat line chart

```
fig = px.line(
    monthly,
    x="Bulan",
    y="Penjualan",
    markers=True,
    title="Tren Penjualan Bulanan"
)

st.plotly_chart(fig, use_container_width=True)
```

px.line() membuat grafik garis. **x** adalah sumbu horizontal, **y** adalah nilai yang diplot. **markers=True** menampilkan titik pada setiap periode. **st.plotly_chart()** menampilkan grafik ke halaman Streamlit.

Mengapa menggunakan fig?

fig adalah objek grafik yang dibuat oleh Plotly. Kita dapat mengubah judul, axis, ukuran, tooltip, dan atribut lainnya sebelum objek tersebut dikirim ke Streamlit.

13. GRAFIK KATEGORI DAN TOP PRODUK

Pada bagian ini kita menerapkan pola yang sama: **groupby** → **sum** → **sort** → **visualisasi**. Perbedaannya adalah dimensi analisisnya.

13.1 Penjualan berdasarkan kategori

```
category_sales = (
    filtered.groupby("Kategori", as_index=False)["Penjualan"]
    .sum()
    .sort_values("Penjualan", ascending=False)
)
```

Data dikelompokkan berdasarkan Kategori, lalu Penjualan dijumlahkan. **sort_values()** mengurutkan hasil berdasarkan Penjualan. **ascending=False** berarti terbesar berada di atas.

Kemudian dibuat bar chart:

```
fig = px.bar(
    category_sales,
    x="Kategori",
    y="Penjualan",
    title="Penjualan Berdasarkan Kategori",
    text_auto=".2s"
)
st.plotly_chart(fig, use_container_width=True)
```

px.bar() digunakan untuk membandingkan nilai antar kategori. **text_auto** menampilkan nilai secara otomatis pada grafik.

13.2 Top produk

```
product_sales = (
    filtered.groupby("Produk", as_index=False)["Penjualan"]
    .sum()
    .sort_values("Penjualan", ascending=True)
    .tail(8)
)
```

Kita mengurutkan dari kecil ke besar lalu mengambil delapan baris terakhir dengan **tail(8)**. Hasilnya adalah delapan produk dengan penjualan terbesar.

```
fig = px.bar(
    product_sales,
    x="Penjualan",
    y="Produk",
    orientation="h",
    title="Top Produk Berdasarkan Penjualan"
)
st.plotly_chart(fig, use_container_width=True)
```

orientation='h' membuat bar horizontal. Ini berguna ketika nama produk panjang sehingga lebih mudah dibaca.

14. DONUT CHART CHANNEL

Pie/donut chart cocok untuk menunjukkan kontribusi setiap channel terhadap total penjualan. Pertama kita membuat ringkasan penjualan per channel.

```
channel_sales = (
    filtered.groupby("Channel", as_index=False)["Penjualan"]
    .sum()
    .sort_values("Penjualan", ascending=False)
)
```

Setiap channel diringkas menjadi total Penjualan. Selanjutnya Plotly membuat pie chart.

```
fig = px.pie(
    channel_sales,
    names="Channel",
    values="Penjualan",
    title="Kontribusi Penjualan per Channel",
    hole=0.45
)
st.plotly_chart(fig, use_container_width=True)
```

names='Channel' menentukan kategori, sedangkan **values='Penjualan'** menentukan ukuran setiap bagian. **hole=0.45** memberi lubang di tengah sehingga pie chart berubah menjadi donut chart.

Catatan analisis:

> Donut chart menunjukkan proporsi, bukan urutan waktu. Untuk tren waktu, gunakan line chart. Untuk membandingkan banyak kategori dengan lebih presisi, bar chart sering lebih mudah dibaca.

15. TABEL DAN DOWNLOAD

Setelah KPI dan grafik, dashboard perlu menyediakan detail transaksi agar pengguna dapat melakukan pengecekan data. Kita juga memberikan tombol download agar hasil filter dapat digunakan kembali.

15.1 Mengurutkan transaksi

```
display_df = filtered.sort_values(
    "Tanggal",
    ascending=False
)
```

Data diurutkan berdasarkan Tanggal dari terbaru ke terlama. Hasilnya disimpan sebagai display_df agar tidak mengubah filtered.

15.2 Menampilkan DataFrame

```
st.dataframe(
    display_df,
    use_container_width=True,
    hide_index=True
)
```

st.dataframe() mengubah DataFrame menjadi tabel interaktif di browser. **use_container_width=True** membuat tabel mengikuti lebar area dashboard. **hide_index=True** menyembunyikan nomor index Pandas agar tampilan lebih bersih.

15.3 Membuat file CSV dari hasil filter

```
csv = filtered.to_csv(index=False).encode("utf-8")
```

to_csv() mengubah DataFrame menjadi format CSV. **index=False** mencegah index Pandas ikut masuk sebagai kolom tambahan. **encode('utf-8')** mengubah teks menjadi format yang dapat dikirim sebagai data download.

15.4 Membuat tombol download

```
st.download_button(
    "■■ Download Data Hasil Filter",
    data=csv,
    file_name="data_penjualan_filtered.csv",
    mime="text/csv"
)
```

data=csv menentukan isi file. **file_name** menentukan nama file yang akan diterima user. **mime='text/csv'** memberitahu browser bahwa file tersebut adalah CSV.

Alur fitur ini:

```
User memilih filter
       ↓
filtered dibuat
       ↓
filtered ditampilkan sebagai tabel
       ↓
filtered diubah menjadi CSV
       ↓
User klik Download
       ↓
data hasil filter tersimpan
```

> Perhatikan bahwa yang di-download adalah **filtered**, bukan seluruh df. Jadi file yang diterima user mengikuti filter yang sedang aktif.

16. MENJALANKAN DASHBOARD FINAL

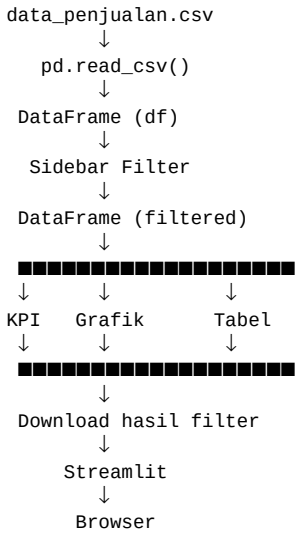
Pastikan app.py dan data_penjualan.csv berada di folder yang sama. Kemudian:

```
conda activate dashboard
cd D:\Dashboard
```

```
python -m streamlit run app.py
```

Buka browser pada **http://localhost:8501**. Jangan menutup Anaconda Prompt selama aplikasi berjalan. Untuk menghentikan server tekan Ctrl+C.

17. MEMAHAMI ALUR PROGRAM SECARA UTUH



Inilah pola utama yang perlu dikuasai: **data** → **proses** → **KPI** → **visualisasi** → **interaksi** → **output**. Jika pola ini dipahami, dataset dapat diganti tanpa harus mengubah konsep dashboard.

18. TROUBLESHOOTING

Error/masalah	Solusi
python was not found	Gunakan Anaconda Prompt lalu conda activate dashboard.
conda is not recognized	Buka Anaconda Prompt dari Start Menu.
No module named streamlit	Aktifkan environment lalu pip install streamlit.
FileNotFoundError	Pastikan app.py dan data_penjualan.csv satu folder.
localhost tidak terbuka	Pastikan Streamlit masih berjalan dan gunakan URL yang muncul.
Port 8501 dipakai	Gunakan URL/port yang ditampilkan terminal.
Perubahan tidak terlihat	Simpan app.py lalu Rerun/refresh browser.

19. CHEAT SHEET

Tujuan	Kode
Aktifkan environment	conda activate dashboard
Masuk folder	cd D:\Dashboard
Jalankan	python -m streamlit run app.py
Stop server	Ctrl+C
Baca CSV	pd.read_csv('data.csv')
Jumlahkan	df['Kolom'].sum()
Nilai unik	df['Kolom'].unique()
Filter	df['Kolom'].isin(pilihan)
Group By	df.groupby('Kolom')['Nilai'].sum()

Tujuan	Kode
KPI	st.metric('Label', nilai)
Line	px.line(data, x='X', y='Y')
Bar	px.bar(data, x='X', y='Y')
Pie	px.pie(data, names='X', values='Y')
Tabel	st.dataframe(df)

20. PENGEMBANGAN LANJUTAN

- Membaca Excel secara langsung.
- Membersihkan data dengan Pandas.
- Menambahkan filter Sales atau Produk.
- Membuat multi-page dashboard.
- Menggunakan database sebagai sumber data.
- Menambahkan CSS, logo, dan branding.
- Membangun dashboard marketing, produksi, finance, atau HR.
- Deployment agar dashboard dapat diakses melalui internet.

PENUTUP

Jangan berfokus menghafal seluruh sintaks. Kuasai pola **data** → **proses** → **KPI** → **visualisasi** → **interaksi** → **insight**. Setelah memahami pola ini, peserta dapat mengganti dataset dan membangun dashboard untuk berbagai kasus bisnis.

Selamat praktik! ■